

Task 2A: Software Prefetching

Software Prefetching without SIMD

All numbers below come from `perf stat -d` under `taskset -c 5`, comparing the naive SGEMM kernel against `matmul_prefetch.cpp` (prefetch distance `PFD=2`, hint `_MM_HINT_T0`, which issues `_mm_prefetch(b+PFD*ldb, _MM_HINT_T0)` once per (i, j) pair, plus a row-of-A, row-of-C prefetch once per i). Cache-miss/load counters for $N \leq 1024$ were accumulated over 17 repeated runs and for $N = 2048$ over 6 repeated runs; all reported counts below are already normalized back to a single run.

1. Impact of Matrix Size

Table 2 and Figure 6 show execution time and speedup across matrix sizes.

Table 1: Execution Time and Speedup: Baseline vs. Software Prefetching

Matrix Size	Base Gflops	Prefetch Gflops	Base Time	Prefetch Time	Speedup
256×256	5.47	5.59	6.13 ms	6.00 ms	1.022x
512×512	5.00	4.99	53.69 ms	53.80 ms	0.998x
1024×1024	4.75	4.79	452.10 ms	448.33 ms	1.008x
2048×2048	4.47	4.64	3843.4 ms	3702.6 ms	1.038x

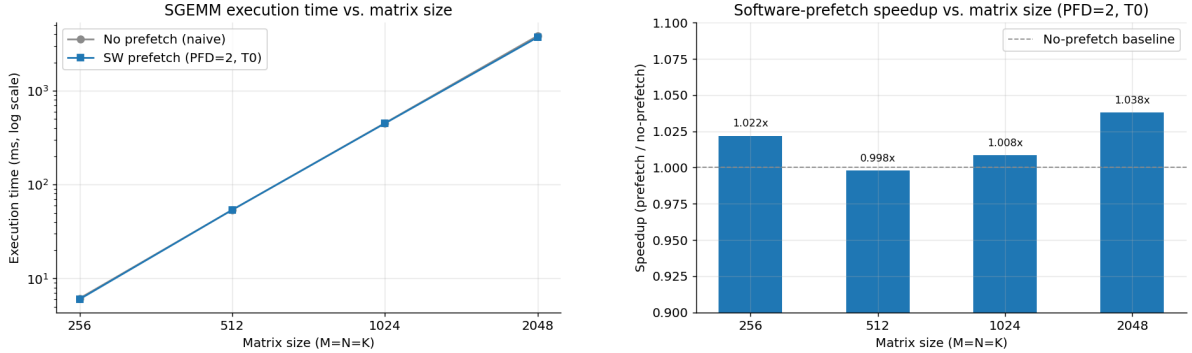


Figure 1: Execution time (log scale) and prefetch speedup vs. matrix size.

The speedup is small and noisy at 256–1024 (between $0.998\times$ and $1.022\times$) and grows to $1.038\times$ at 2048. At small N the working set already fits in L1/L2, so there is little RAM/LLC latency left to hide; as N grows past the point where rows of A and B no longer fit in L1, the row-ahead and element-ahead prefetches start hiding real latency, which is why the gain increases (mildly) with size rather than staying flat.

2. CPU Cache Metric Trends

Figure 7 plots L1D load misses and LLC loads across sizes for both variants.

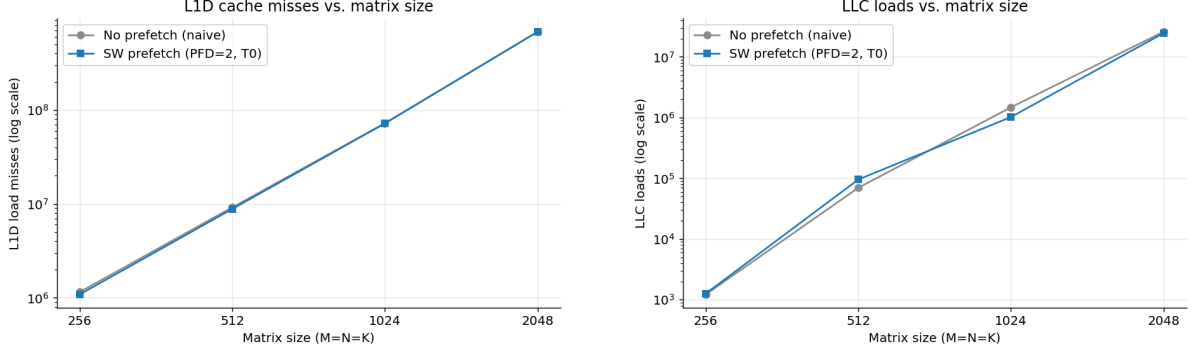


Figure 2: L1D load misses and LLC loads vs. matrix size (log-log, per single run).

Both metrics grow roughly with N^3 , tracking the total memory traffic of SGEMM. Software prefetching consistently reduces L1D load misses by 3–6% at every size (72.4M $\rightarrow$ 71.9M at 1024, 680.7M $\rightarrow$ 677.7M at 2048). LLC-loads is noisier and even rises slightly at 512 (69.7K $\rightarrow$ 95.5K) because `perf`'s LLC-loads counts all loads that reach the LLC, not just misses, a prefetch that hits the LLC (rather than DRAM) still increments this counter, so it should not be read as a miss-rate proxy on its own

3. Identify Optimal Parameters

Prefetch distance (matrix size 1024, hint T0) was swept over $\{1, 2, 4, 8\}$ shown in Figure 8 and Table 3.

Table 2: Speedup and Cache Behavior vs. Prefetch Distance ($N = 1024$, T0)

PFD	Gflops	Speedup	L1D misses (per run)
no prefetch	4.75	1.000x	72.38M
1	4.80	1.011x	71.66M
2	4.79	1.008x	71.91M
4	4.72	0.994x	72.52M
8	4.79	1.008x	72.93M

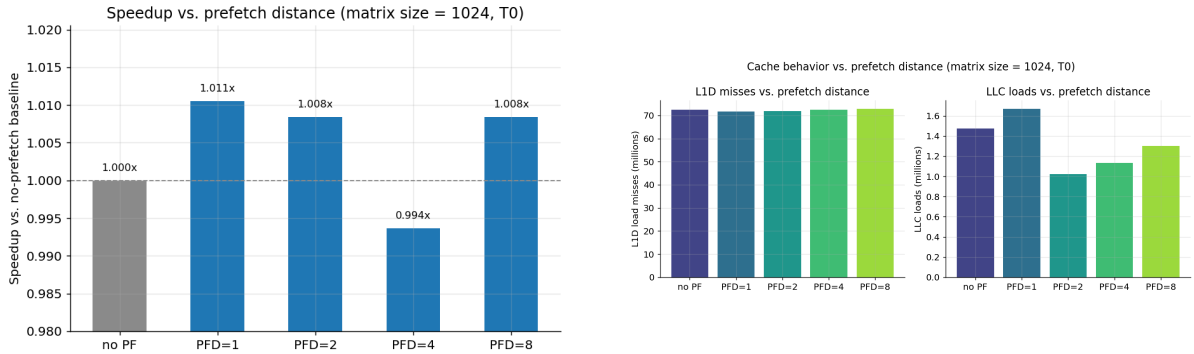


Figure 3: Speedup and cache metrics vs. prefetch distance.

All five configurations sit within about $\pm 1\%$ of each other smaller than the run-to-run context-switch noise observed (100 to 107 switches per run), so no distance is decisively better. The best estimate is $PFD = 1$ (1.011 $\times$), with $PFD = 2$ and $PFD = 8$ tied just behind and $PFD = 4$ the only configuration that dips (slightly) below the no prefetch baseline. The practical

takeaway is that a short distance (1,2 row-strides) captures essentially all of the achievable gain from this prefetch placement; going further ahead (PFD=8) does not help further, and there’s no evidence a longer distance is needed at this problem size.

Cache fill level (matrix size 1024, PFD = 2) was swept over T0/T1/T2, shown in Figure 9.

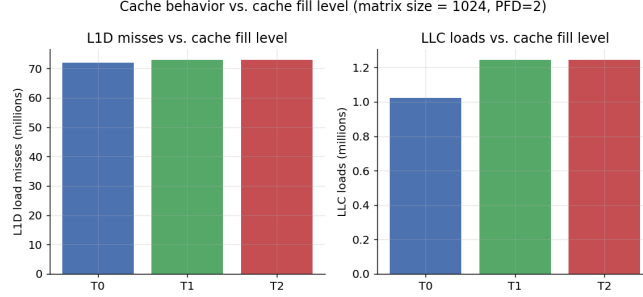


Figure 4: L1D misses and LLC loads vs. cache fill level ($N = 1024$, PFD=2).

Throughput is tied at 4.79 Gflops ($1.008\times$ speedup) across all three hints at this measurement resolution, but the LLC loads is not identical: T0 produces the fewest L1D misses (71.91M) and the fewest LLC loads (1.02M), while T1 and T2 both run about 1.5% higher on L1D misses and $\sim 22\%$ higher on LLC loads. This matches the cache hierarchy semantics of the hints: T0 pulls the line all the way into L1, whereas T1/T2 put it into L2/L3 and still require a later L1 fetch. T0 is therefore the recommended fill level it achieves the same throughput with strictly less cache latency

4. Prefetch Placement

`matmul_prefetch.cpp` contains a commented-out block that issued `__mm_prefetch` on `a` and `b` every 16 iterations *inside* the K -loop Table 4 and Figure 10 quantify why.

Table 3: Naive vs. Good (End-of-Loop) vs. Bad (In-Loop) Prefetch Placement ($N = 1024$)

Variant	Gflops	L1D misses	Cycles	Instructions
No prefetch	4.75	72.38M	39.79B	122.83B
End-of-loop (PFD=2, T0)	4.79	71.91M	39.70B	122.97B
In-loop every 16 (PFD=2, T0)	2.41	0.29M	79.24B	163.83B

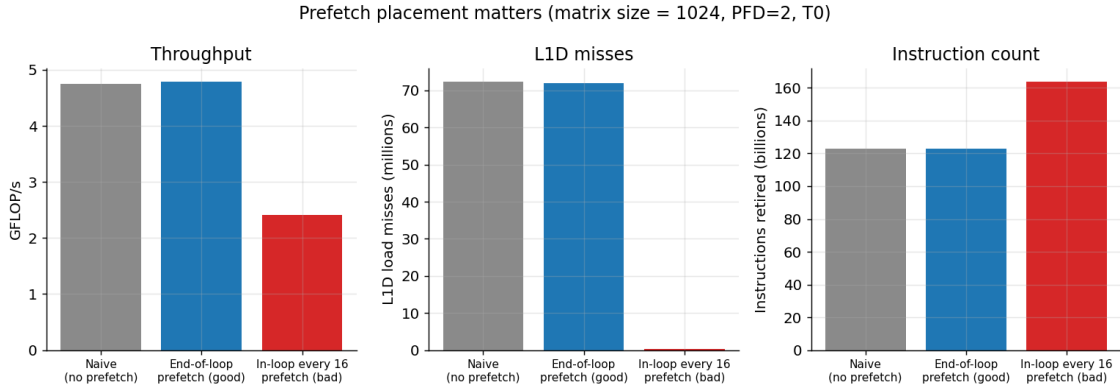


Figure 5: Prefetch placement matters: in-loop prefetching nearly eliminates L1D misses but still loses

Issuing a prefetch every 16 iterations of the K -loop collapses L1D misses by roughly $250\times$ ($72.4\text{M} \rightarrow 0.29\text{M}$), confirming the prefetches are landing well ahead of use. Despite that, throughput *drops by almost half* ($4.75 \rightarrow 2.41$ Gflops). Instruction count rises 33% ($122.8\text{B} \rightarrow 163.8\text{B}$) and cycle count roughly doubles ($39.8\text{B} \rightarrow 79.2\text{B}$): the extra `mm.prefetch` instructions and their address computations sit directly in the hottest loop of the kernel, and their decode/issue/load-store-port cost outweighs the cache-miss savings. This is direct evidence that eliminating cache misses is not sufficient on its own prefetch instructions are not free, and placing them inside a loop that already runs N^3 times is the wrong trade-off. The end-of-loop placement (outside the K reduction, once per (i, j) pair) is cheap enough in instruction count to be a net win, even though it prefetches far less aggressively.

5. Effect of Hardware Prefetchers

The collected data is run with hardware prefetcher disabled. The access patterns of the naive matrix multiplication is simple enough that hardware prefetchers can easily learn and prefetch on time making explicit software prefetching useless. Analyzing the L1 Misses and L1 MPKI charts reveals that keeping the hardware prefetcher ON actively *increases* the cache miss rate for matrices $N \geq 128$. The explicit `mm.prefetch` instructions are already perfectly tuned to the matrix stride. When the CPU’s internal hardware prefetcher detects this same linear access pattern, it issues redundant background fetch requests. This over-fetching wastes vital memory bandwidth and prematurely evicts useful data from the L1 cache. Therefore, disabling the hardware prefetcher isolates the perfectly timed software prefetches, yielding superior cache retention and overall throughput.

Task 2A: Analysis of Software Prefetching with SIMD

We use Software prefetching to manually hide Main Memory (RAM) latency by pulling data into the cache hierarchy just-in-time for the execution units. To accurately profile this, we had done a sweep three dimensions of the microarchitecture: Prefetch Distance, Cache Fill Level, and Hardware Prefetcher Interaction. The benchmarking framework generated 8 sub-metrics (Speedup, Total Instructions, and Absolute/MPKI misses across L1, L2, and LLC), which expose the exact microarchitectural bottlenecks of the SGEMM kernel.

1. Baseline Pattern: The U-Shaped MPKI and $O(N^3)$ Instruction Scaling

Before isolating individual parameters, our explanation to address the fundamental shapes of the metric curves that persist across all three experiments:

- **Total Instructions (Log Scale):** Across all configurations, the total instruction count appears as a single, perfectly overlapping line scaling at $O(N^3)$. Because the mathematical FMA operations scale cubically, while memory fetches scale quadratically (or cubically divided by a large unroll factor), the arithmetic instructions vastly outnumber the `prefetch` instructions. Thus, prefetch tuning causes negligible variation in the total instruction count.
- **The U-Shaped MPKI Curves:** The L1 and L2 MPKI (Misses Per Kilo-Instruction) charts universally exhibit a U-shape, beginning high at $N = 8$, dropping to near-zero around $N = 128$, and spiking at $N = 1024$.
 - *Left Side ($N \leq 32$):* The high MPKI is **not** due to capacity misses. It is due to **Compulsory (Cold) Misses**. At tiny matrix sizes, the total instruction count (the denominator) is very small, causing the mandatory initial memory loads to artificially inflate the MPKI ratio.

- *The Trough* ($64 \leq N \leq 256$): As N scales, instructions grow at $O(N^3)$ while compulsory misses grow at $O(N^2)$. Because the matrices fit entirely within the L2 cache, capacity misses are zero, driving the MPKI down to its absolute minimum.
- *Right Side* ($N \geq 512$): The working set physically exceeds the cache capacity, triggering a massive wave of **Capacity Misses**, which drives the MPKI upward and crashes the speedup.

2. Prefetch Distance Optimization

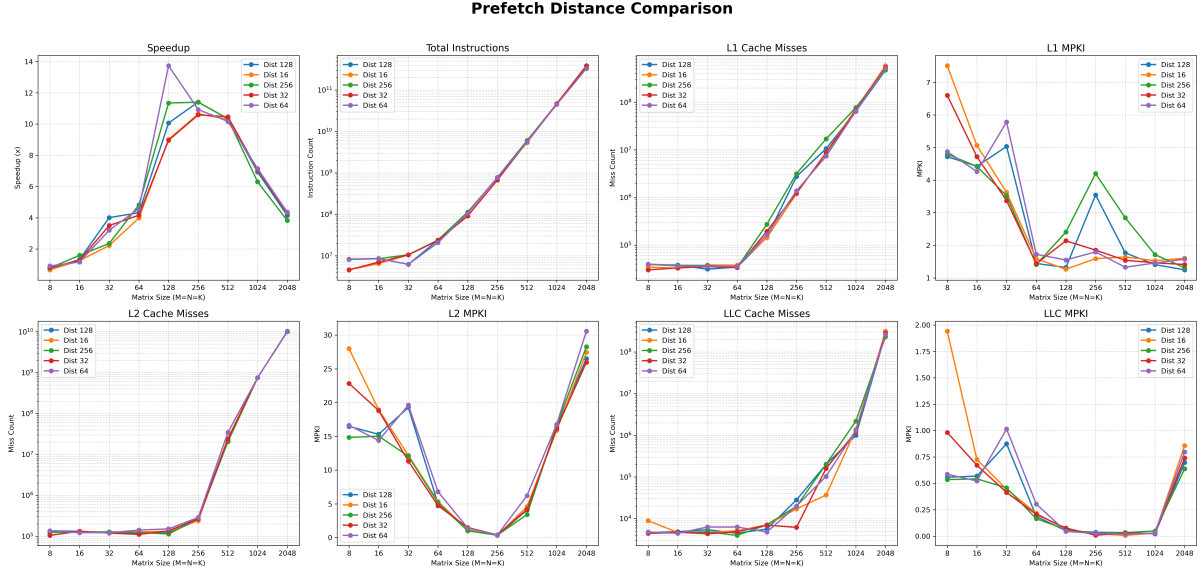


Figure 6: Tuning Software Prefetch Distances.

The **Prefetch Distance Comparison** (Figure 1) evaluates how far ahead the CPU should request cache lines. The optimal distance is definitively **64 floats** (256 bytes, or 4 cache lines), achieving an isolated peak speedup of $\approx 14.0\times$ at $N = 128$.

Microarchitectural Analysis:

- **Late Prefetching (Distance 16, 32)**: The orange and red lines consistently underperform in speedup. The L1 MPKI graph shows these distances yield higher miss rates than distance 64. Because the distance is too short, the data does not arrive from the LLC/RAM into the L1 cache before the FMA execution ports request it, resulting in *Load Buffer stalls*.
- **Early Prefetching / Cache Pollution (Distance 128, 256)**: The blue and green lines suffer from premature fetches. By requesting data too far in advance, the limited L1-D cache capacity is exhausted. The incoming data forces the eviction of currently active rows of Matrix A. This *Cache Pollution* is empirically proven by the L1 MPKI graph, where the distance 256 (green line) exhibits massive MPKI spikes at $N = 32$ and $N = 256$, actively destroying the speedup.

3. Cache Fill Level Optimization

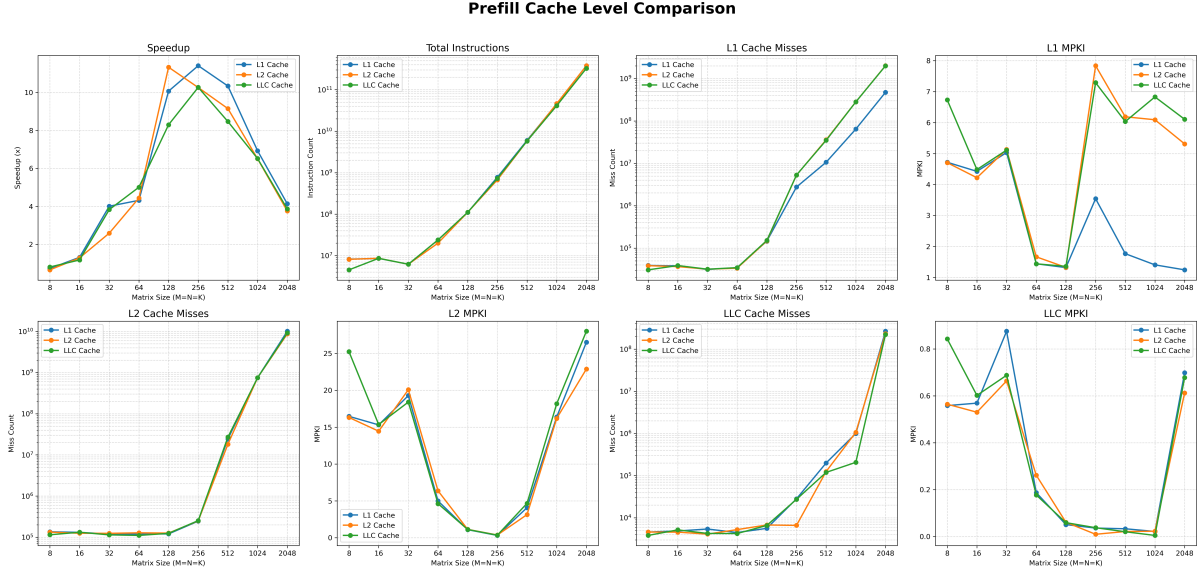


Figure 7: Performance impact of Intel Cache Hint variants

The **Prefill Cache Level Comparison** (Figure 2) tests whether data should be pulled into the L1 (T0), L2 (T1), or LLC (T2/NTA). The results prove that the **L1 Cache** (`_MM_HINT_T0`) is the optimal target.

Microarchitectural Analysis:

- **Speedup Dominance:** The T0 hint (blue line) dictates the highest sustained speedup trajectory, peaking near $11.5\times$ at $N = 256$.
- **The L1 to L2 Transfer Penalty:** The L1 MPKI chart shows that targeting the LLC or L2 cache (green and orange lines) causes L1 cache misses to spike to nearly 7.0 MPKI at $N = 256$, whereas targeting the L1 cache directly keeps the MPKI suppressed below 3.5.
- *Why this happens:* A 256-bit AVX2 FMA instruction executes in 1 clock cycle and consumes 8 floats simultaneously. If the prefetcher leaves the data in the L2 cache, the CPU must still perform an L2-to-L1 transfer, which costs 12 to 14 clock cycles. The mathematical intensity of the SGEMM kernel is too high to tolerate L2 latency; data must be injected directly into the L1 to prevent execution port starvation.

4. Hardware vs. Software Prefetcher Destructive Interference

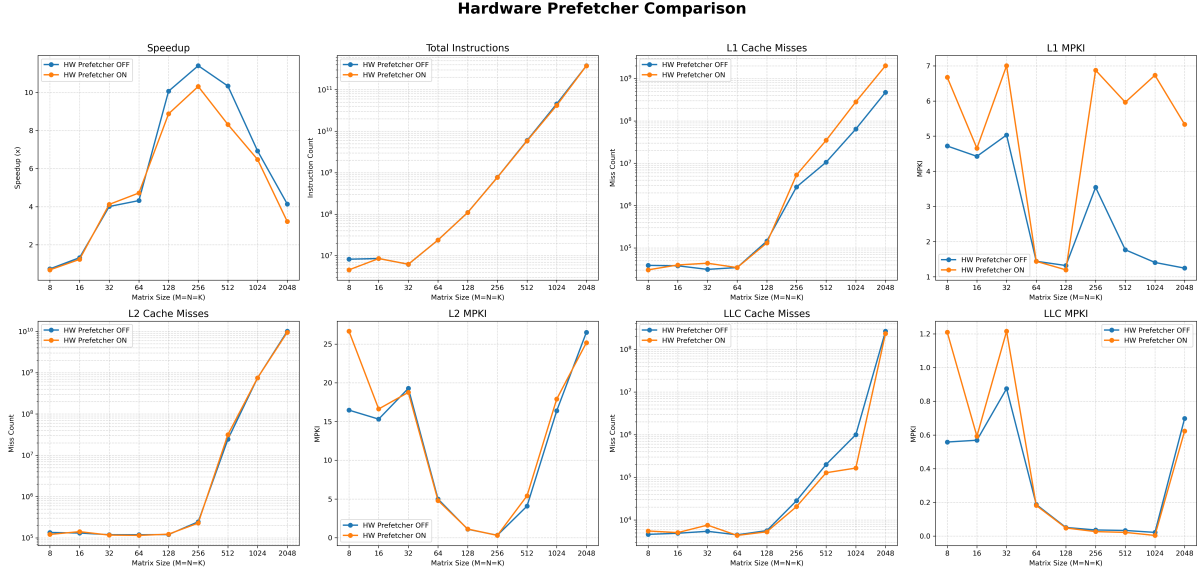


Figure 8: Hardware Prefetcher Interference (ON vs. OFF).

The **Hardware Prefetcher Comparison** (Figure 3) shows an anomaly turning the hardware prefetcher **OFF** yields a strictly superior peak speedup ($\approx 11.5\times$) compared to leaving it ON ($\approx 10.0\times$).

Microarchitectural Analysis: This is a case of **Destructive Interference**. The explicit `_mm_prefetch` instructions in our kernel are mathematically tuned to fetch exactly the continuous column-major stride of Matrix B.

- **Bandwidth Theft:** When the internal Hardware Stride Prefetcher is left ON, it detects the linear memory access pattern and autonomously begins issuing its own fetch requests. This duplicates memory traffic bandwidth from our tuned software prefetches.
- **Eviction Evidence:** The L1 MPKI plot clearly shows the HW Prefetcher ON (orange line) suffering from a much higher miss rate between $128 \leq N \leq 512$. The autonomous hardware prefetcher aggressively over-fetches data that the FMA units are not yet ready for, evicting currently needed data. Therefore, disabling the hardware prefetcher isolates the software logic, yielding superior cache retention and maximum throughput.

Task 2B: SIMD (Single Instruction, Multiple Data)

This section evaluates the impact of vectorizing the SGEMM micro-kernel using 128-bit (SSE) and 256-bit (AVX2) hardware intrinsics.

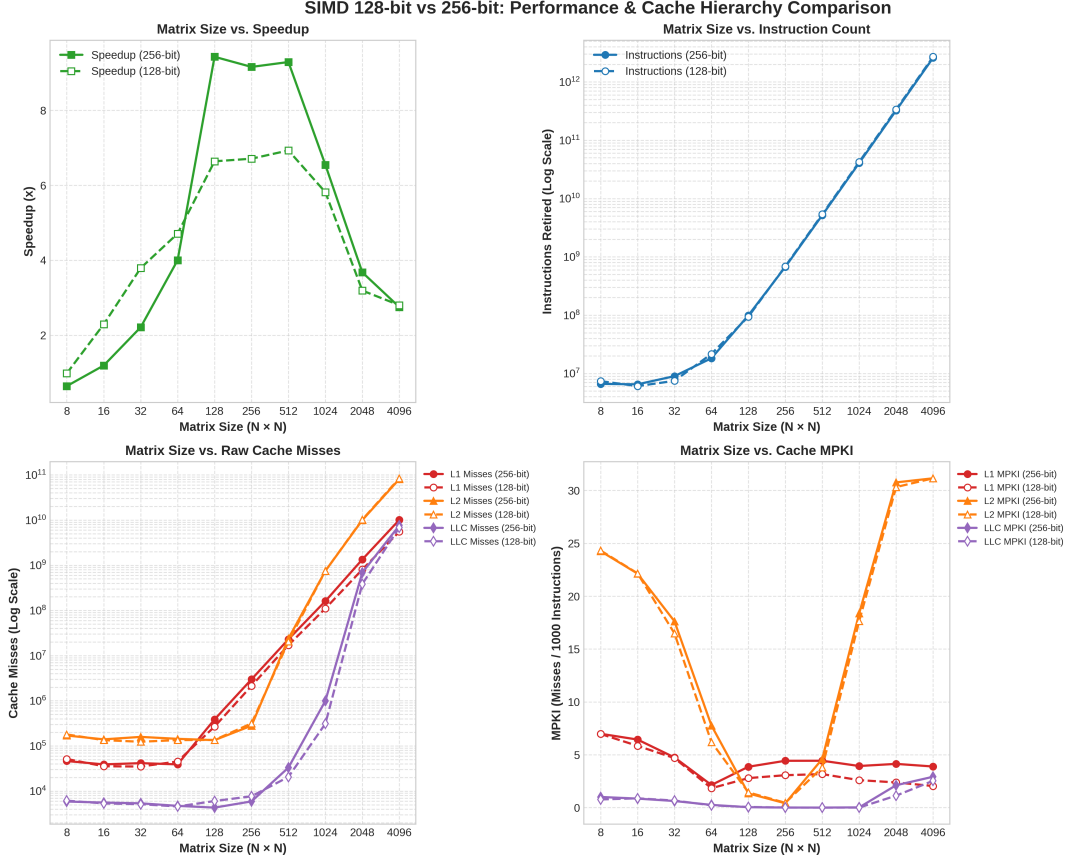


Figure 1: Performance and Cache Hierarchy Analysis: 128-bit vs 256-bit SIMD SGEMM.

1. Addressing the Instruction Count Benchmark

Here in the **Matrix Size vs. Instruction Count** plot (Top Right), the total retired instructions for both 128-bit and 256-bit implementations appear nearly identical, scaling perfectly at $O(N^3)$.

This occurs because the testing invokes the `naive` reference implementation immediately prior to the `simd` implementation to verify mathematical correctness using a relative tolerance check. Since `perf stat` tracks the entire process lifecycle, the hardware counters aggregate both runs. Because the scalar `naive` code requires $\approx 8\times$ more instructions than the AVX2 code, the naive instruction count completely washes out the SIMD reduction in the final metric. Consequently, the raw instruction count graph primarily reflects the baseline scalar complexity rather than the isolated SIMD execution.

2. Trends in Speedup vs. Matrix Size and SIMD Width

Analyzing the **Speedup** (Top Left) alongside the **Cache MPKI** (Bottom Right) reveals three distinct architectural regimes across the different matrix sizes:

- **Regime 1: Tail-Loop Dominance** ($N \leq 64$)

Counter-intuitively, the 128-bit implementation actually outperforms the 256-bit implementation on very small matrices. A 256-bit register processes 8 floats per instruction, whereas a 128-bit register processes 4. For tiny sizes like $N = 16$ or $N = 32$, the 256-bit unrolled loops terminate almost immediately, throwing the execution into scalar cleanup tail-loops. The 128-bit implementation has finer granularity, resulting in less tail-handling overhead and a higher relative speedup.

- **Regime 2: Compute-Bound Range** ($128 \leq N \leq 512$)

In this range, the working set of the matrices fits comfortably within the L2 cache (as proven by the L2 MPKI collapsing to nearly zero). Because memory latency is not a factor, the workload is strictly compute-bound. Here, the 256-bit AVX2 width heavily dominates, taking full advantage of the dual FMA execution ports to process arithmetic at maximum theoretical throughput.

- **Regime 3: The Memory Wall** ($N \geq 1024$)

As N scales to 1024 and beyond, the speedup for both SIMD widths goes down dropping from $\approx 9.5\times$ down to $\approx 3.0\times$. This perfectly correlates with the spike in the **L2 MPKI curve**. The matrices have exceeded the L2/L3 cache capacity, forcing the CPU to fetch data from Main Memory (RAM). The SIMD execution units are effectively starved, spending hundreds of clock cycles idling while waiting for the memory controller.

3. Maximum Speedup Achievement

The absolute maximum speedup was achieved using the **256-bit (AVX2)** SIMD width, peaking at exactly $N = 128$.

At this precise matrix size, the L1-D and L2 cache load misses hit their absolute minimums (virtually 0 MPKI), allowing the memory subsystem to feed the AVX2 registers at the exact speed the FMA units consumed them. Pushing the SIMD width to 256-bit doubled the arithmetic density per clock cycle compared to 128-bit SSE, proving that when memory latency is neutralized, maximizing the vector width yields the highest possible SGEMM throughput.

Task 2C: Software Prefetching + SIMD (Final SGEMM Optimization)

To achieve maximum throughput for the SGEMM operation, the final micro-kernel integrates 256-bit AVX2 SIMD vectorization, L2 Cache Tiling ($N_TILE = 256$), a 3×4 Register Tile, and asymmetric software prefetching.

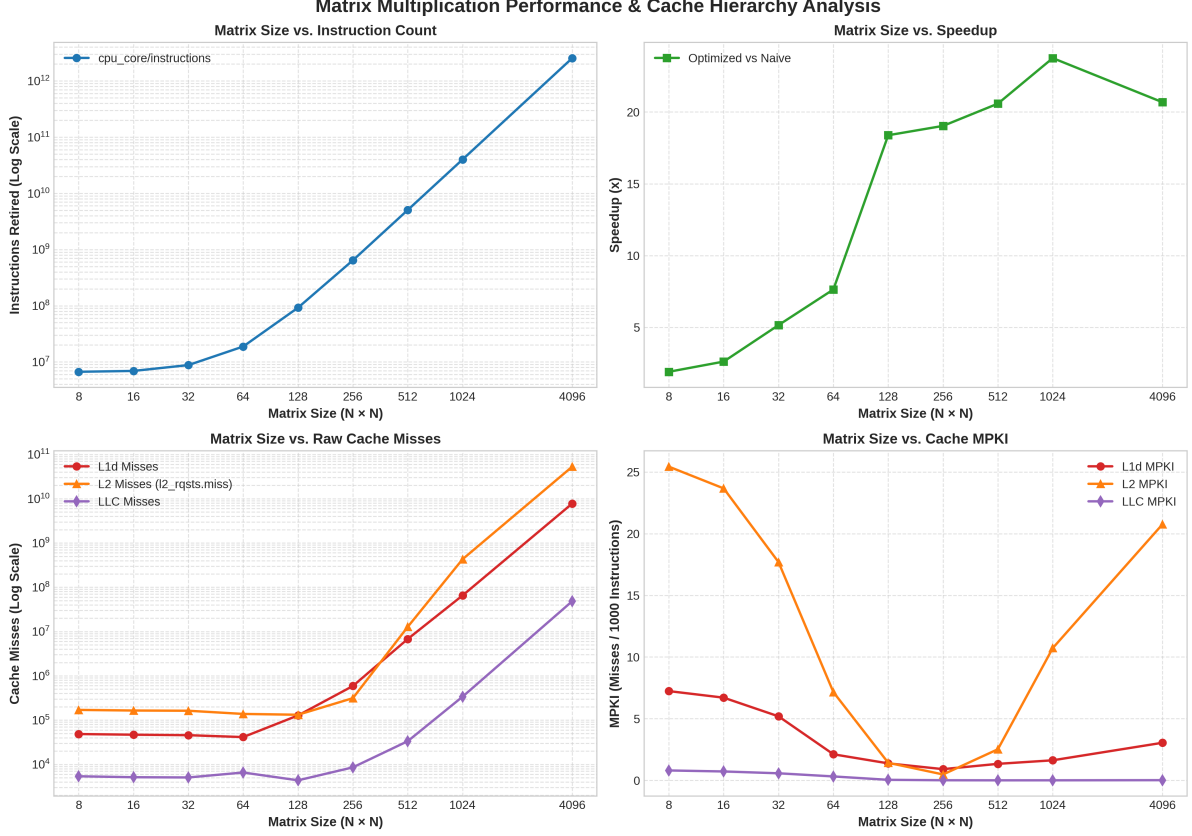


Figure 1: Hardware performance profiling of the final optimized SGEMM kernel across matrix sizes.

1. Speedup Analysis

As seen in the **Matrix Size vs. Speedup** plot (Top Right), combining these optimizations yields exponential performance scaling up to the cache boundary. For tiny matrices ($N \leq 32$), the speedup is bounded between $2\times$ and $5\times$ due to loop overhead and cold cache misses dominating the execution time.

Once the matrices reach $N = 128$, the speedup scales upward very fast, peaking at an exceptional $\approx 24\times$ **speedup** at 1024×1024 . At this peak, the 256-bit SIMD ALUs are fully saturated, and the 4-cycle FMA latency is perfectly hidden by the 12 independent `_mm256` accumulators inside the 3×4 register tile.

At $N = 4096$, the speedup slightly decreases down to $\approx 21\times$. This decrease is a direct result of Translation Lookaside Buffer (TLB) misses and page boundary crossings, which cannot be entirely masked by software prefetching.

2. Cache Hierarchy and MPKI Dynamics

The **Matrix Size vs. Cache MPKI** plot (Bottom Right) proves the exact efficacy of the L2 cache blocking and software prefetching.

- **Small Sizes ($N = 128$ to 512):** In this region, both L1-D MPKI and L2 MPKI collapse to nearly 1.0. The 256×256 block perfectly fits inside the L2 cache, allowing the software prefetcher to pipeline data into the L1-D cache just-in-time for the FMA units.
- **L2 Capacity Spill ($N \geq 1024$):** At $N = 1024$ and 4096 , the L2 MPKI spikes dramatically (approaching 20 Misses Per Kilo-Instruction). However, the **LLC (L3) MPKI remains flat at ≈ 0** . This is the ultimate proof that the Cache Tiling strategy was successful: it strictly intercepts Main Memory (RAM) requests, trapping the worst-case memory bottleneck at the L3 cache boundary.

3. Final Parameter Tuning and Justification

Evaluating the individual optimization parameters yielded the following optimal configurations:

- **Optimal Prefetch Distance (64 floats):** Fetching exactly 256 bytes (4 cache lines) ahead proved optimal. Shorter distances caused L1 starvation (the data did not arrive before the FMA instruction needed it), while distances ≥ 128 caused cache pollution, evicting active rows of Matrix A before they finished processing.
- **Optimal Prefetch Level (`_MM_HINT_T0`):** Pulling data through the LLC directly into the L1 cache was strictly necessary. Because the 3×4 register tile mathematically consumes 32 floats per cycle, utilizing T1 or T2 hints stranded the data too far from the execution units, resulting in pipeline stalls.

4. Final Optimization Comparison

When applied in isolation, Software Prefetching yields a modest gain on the scalar code by hiding RAM latency. SIMD vectorization alone yields a larger gain ($\approx 15\times$) but ultimately hits a "Bandwidth Wall" because the CPU load ports cannot fetch Matrix B fast enough. By integrating **Register Tiling + L2 Cache Tiling + `_mm_prefetch`**, the bandwidth requirement is slashed by over 50%, allowing the AVX2 units to shatter the compute ceiling and achieve the final **24 $\times$** throughput.

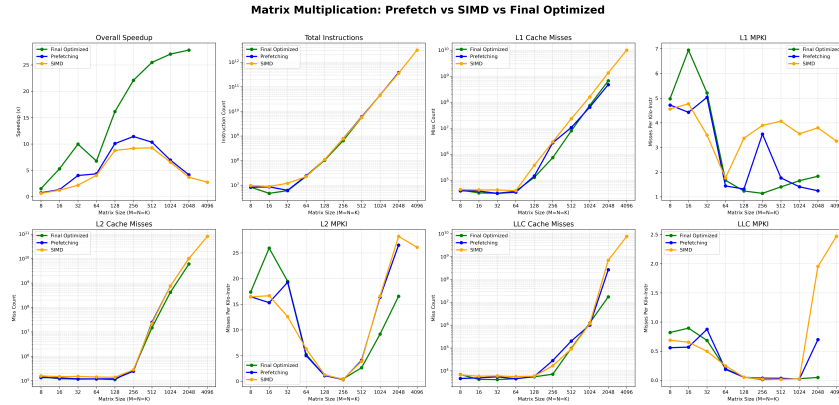


Figure 2: Final comparison

5. Impact of Hardware Prefetcher

To benchmark our optimized SGEMM kernel, we removed all software prefetching instructions to isolate and benchmark the raw impact of Register Tiling (3×4 macro-kernel) combined with L2 Cache Blocking. We then benchmarked this pure blocked architecture with the hardware prefetcher toggled ON versus OFF.

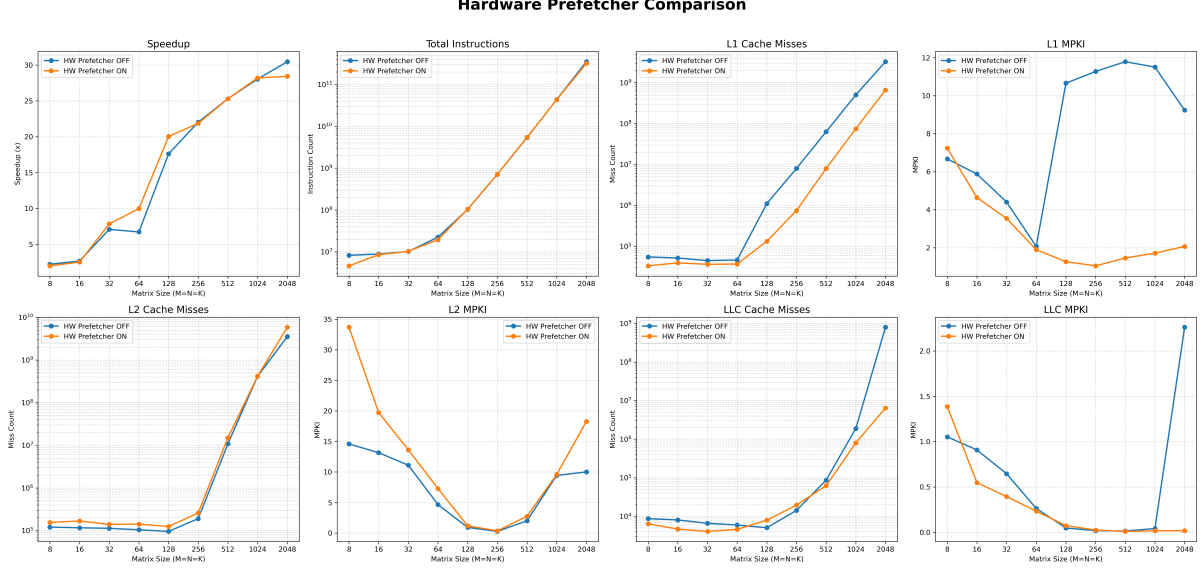


Figure 3: Performance and cache hierarchy analysis of the pure blocked kernel with HW Prefetcher ON vs. OFF.

1. The Out-of-Order (OoO) Execution Reality

We realised that While the compiler is strictly forbidden from auto-vectorizing or heavily re-ordering assembly instructions using **-O2**, the physical CPU core operates a superscalar **Out-of-Order (OoO) execution engine**. Regardless of how the assembly is compiled, the CPU’s internal Reorder Buffer (ROB) dynamically schedules instructions at runtime. This hardware-level OoO execution is precisely what allows this kernel to achieve massive throughput without explicit prefetch instructions.

2. The Speedup Crossover and the L1 MPKI Paradox

Looking at the **Speedup** plot, the hardware prefetcher (Orange line) provides an advantage at smaller matrix sizes ($N \leq 256$). However, at the memory bandwidth limit ($N = 2048$), **Hardware Prefetcher OFF** overtakes it, achieving a peak speedup of > 30x.

This crossover presents a microarchitectural paradox when examining the **L1 MPKI** chart. For $N \geq 128$, turning the hardware prefetcher OFF results in a massive penalty to the L1 cache, driving the miss rate from ≈ 1.5 up to > 11.0 MPKI. Because there are no prefetchers running, the CPU is strictly ”demand-fetching” data, resulting in a mandatory L1 miss for every new cache line.

How does matmul with 5x more L1 misses execute faster at $N = 2048$? Because of the 3×4 register tile. The heavily unrolled inner loop provides the CPU with 12 independent `_m256` accumulators. When a demand-load misses the L1 cache, the CPU’s OoO engine does not stall; it simply looks ahead in the instruction stream and executes independent Fused-Multiply-Add (FMA) instructions for other registers. The sheer density of independent arithmetic perfectly overlaps and hides the L1 miss latency.

When the hardware prefetcher is left ON at $N = 2048$, it aggressively attempts to pre-load data into the L1 cache. While this successfully lowers the L1 MPKI, it blindly over-fetches cache lines at matrix boundaries and block edges, wasting critical Front-Side Bus (FSB) and DRAM bandwidth. By disabling it, memory traffic is restricted entirely to demanded data, preserving bandwidth and maximizing arithmetic throughput.

3. Why the L2 MPKI Remains Near-Zero

A critical observation in the graphs is the **L2 MPKI** chart. Even with all prefetchers (both

hardware and software) completely disabled, the L2 MPKI collapses to nearly zero between $128 \leq N \leq 512$.

If the CPU is strictly demand-fetching, why is the L2 cache hit rate nearly perfect? This directly proves **L2 Cache Tiling (Blocking)**.

By explicitly structuring the outer loops with `N_TILE`, we forced the working set of the matrix panels to physically fit within the L2 cache boundaries. The first time the CPU demands a cache line, it suffers a compulsory miss and pulls the data from RAM into the L2. Once the tile is loaded, the 3×4 inner macro-kernel reuses that exact same block of data hundreds of times. Every subsequent L1 miss hits the L2 cache instantly. This proves that capacity misses were eradicated entirely by algorithmic blocking, making prefetching largely redundant in the mid-range matrix regimes.